STORAGE



IO Prioritization for Optimized Command Handling

Anthony Constantine *Distinguished Member of Technical Staff at Micron Technology*

Yuliya Tarnikova *Staff Software Engineer at Google*

Chaitanya Kulkarni *Senior Director at NVIDIA*

Dan Helmick *Principal Architect at Samsung*

Panel Discussion



Anthony Constantine

DMTS at Micron Technology



Yuliya Tarnikova

Staff Software Engineer at Google



Chaitanya Kulkarni

Senior Director at NVidia



Dan Helmick

Principal Architect at Samsung

Agenda

- Overview – Anthony
- Problem and use case (Hyperscalar view) – Yuliya
- Problem and use case (SSD/HDD/Host SW) – Chaitanya
- SSD Implementation – Dan
- Group discussion and Q&A

IO Prioritization Overview

- Ideal World: SSDs see a single workload and can respond when expected
- Reality: SSDs see several workloads across many queues and there are different latencies depending on the transaction.
- To address reality, the idea of the NVMe® IO prioritization feature came about:
  - The basic goal is to have a lower and more predictable latency for higher priority I/O transactions while still progressing with the remaining bulk work.
- This is proposed in NVM Express® as TP4221 IO Prioritization.
  - The intent of this proposal is to introduce mechanisms within NVMe technology that let's the host set parameters relating to this prioritization.

A super high-level view of how this would work

- The host set a priority per Submission Queue
  - Very similar to how WRR queuing is done today
  - Grows the number of priority classes

- Each priority level would have a “duration limit”
  - This thought has evolved over the course of investigating how to implement.
  - This is similar to Command Duration Limits (CDL) that is done for HDDs

- Device meets priorities based on the host configured values
  - Handling missed deadlines (depending on priority class)
  - Reporting mechanisms

Google point of view: Mixed-Workload Challenge

Challenge

Diverse workloads with different latency sensitivity
Background workloads block critical user-facing reads, causing unpredictable latency spikes.

What breaks today

- ***Rate limiter:*** *focus on Bandwidth and IOPS, not latency.*
- ***Weighted Round Robin:*** *Priorities do not propagate end-to-end and ignore I/O cost.*

Historical approach: *Vendor specific logic in the SSD.*

Desired device behaviour

- ***Single configuration*** *across changing workloads; no priority inversion.*
- *Prioritize higher-priority I/O over short bursts, while maintaining baseline service guarantees for all levels.*

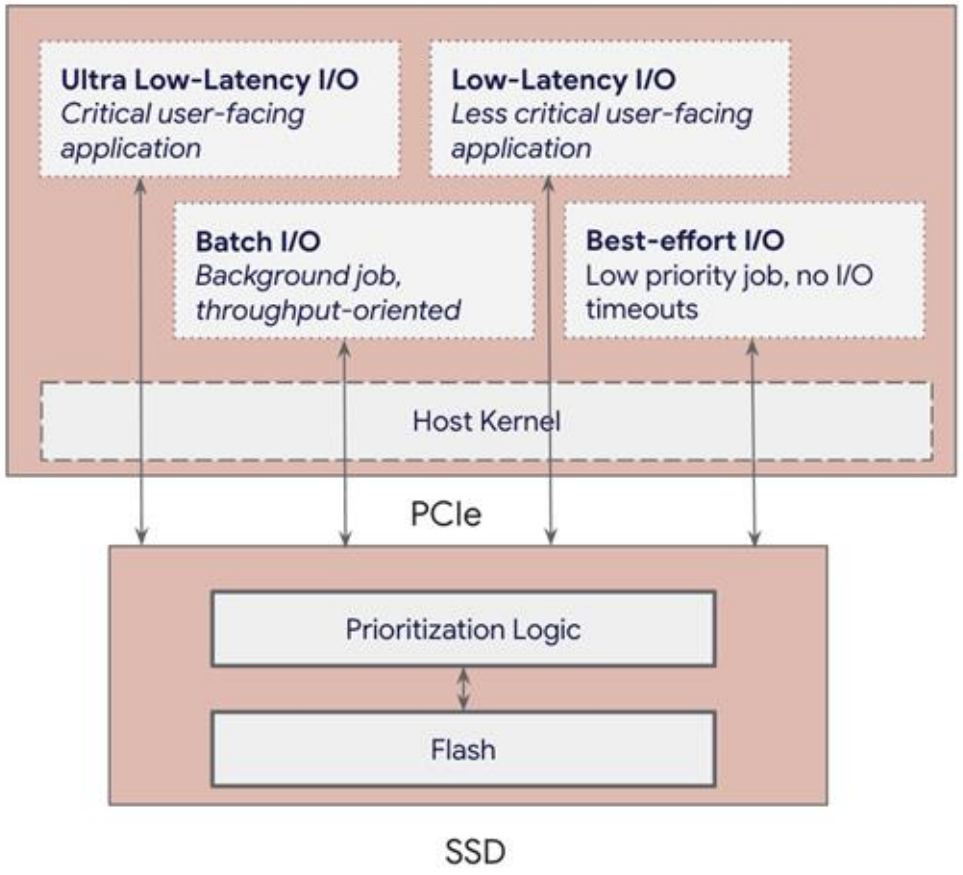
How we will measure success

- ***Performance Benchmarks:*** *Compare latency profiles for high-priority classes with and without the feature.*
- ***Speed of Light Target:*** *Achieve the latency profile of running with lower-priority I/O absent.*

Google point of view: Use Cases

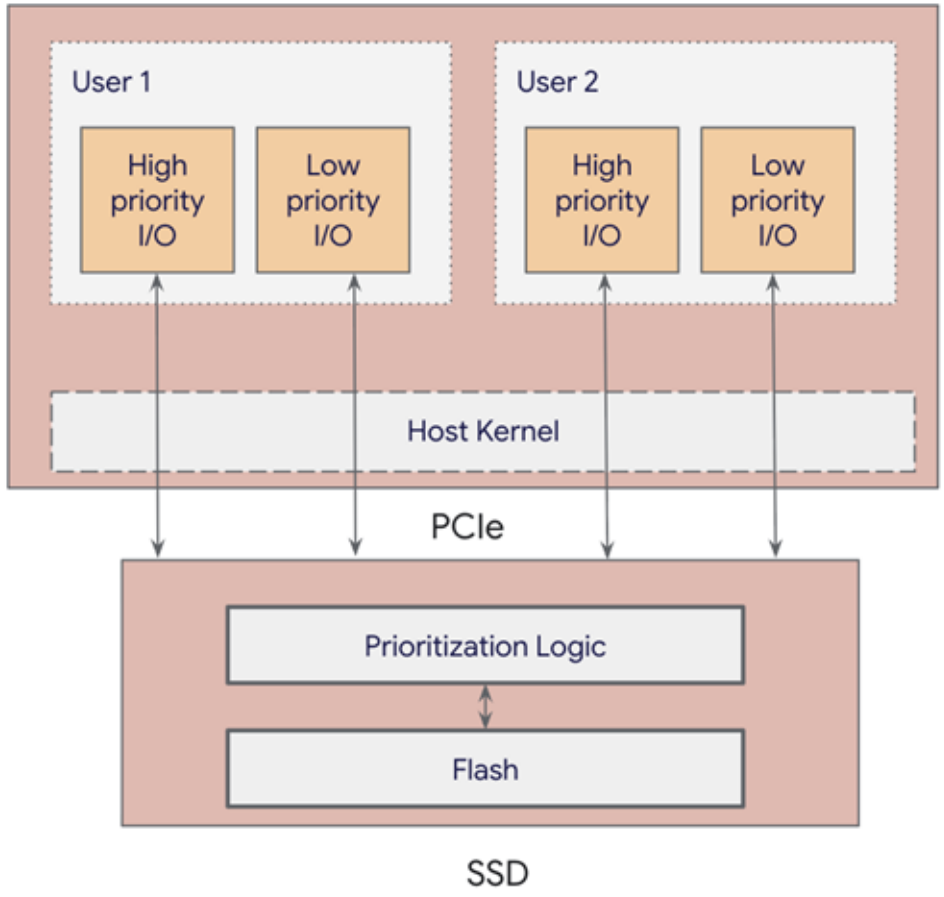
Main use-case:

Diverse storage applications sharing an SSD

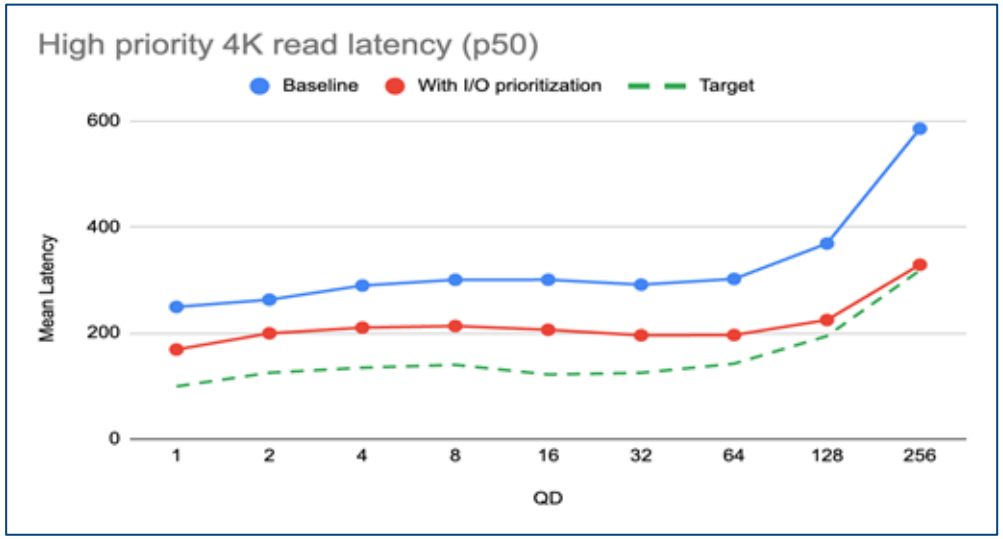


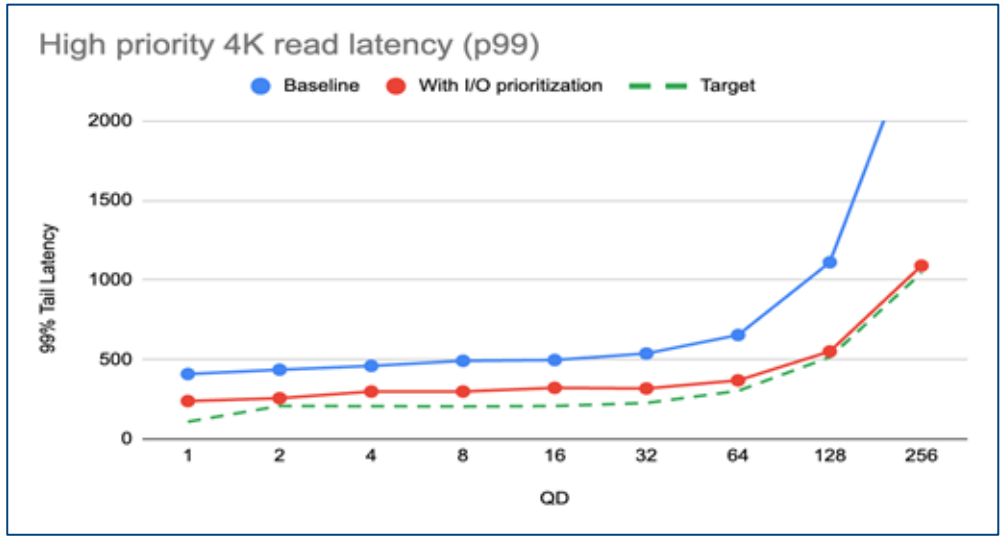
Future opportunities:

Diverse applications within multi-tenant environment



Google point of view: Experiments & Learnings





Significant latency improvements

- *NAND-level arbitration via media time*
- *Latency drops: ~20% at p50, ~50% at p99*
- *Next step: End-to-end prioritization*

Implementation: Weighted Fair Queuing (WFQ)

- *Time-based arbitration (resource consumption vs. command count)*
- *Immediate dispatch for high-priority I/O and guaranteed progress across all priority classes*
- *Consistent scaling despite I/O floods and changing workloads*

NVIDIA point of view : Use case

- Modern HPC, AI/ML training, and GPU clusters require fine-grained, concurrent LBA access within a single NVMe® namespace to ensure efficient parallel I/O.
  - This shift to finer-grained accesses boosts concurrent throughput. As a result, highly parallel, GPU-initiated access becomes more practical.
- This leads to workloads dominated by small-size I/Os, where large, high-priority I/Os end up waiting behind deep queues of small-size, medium-priority I/Os.
- In the existing NVMe architecture, there is no way to boost high-priority I/Os that are waiting behind small-size, medium-priority I/Os, leading to higher latency for high-priority I/Os.

Nvidia point of view : Use case

- New proposed mechanism solves this problem where Host S/W can specify the Command Durations limits and tag appropriate I/Os with high priority while continuing the medium priority I/Os.

Linux kernel maintainer’s view: Use case

- The Linux kernel lacks support for NVMe® I/O priority.
- The Linux kernel interface needs to be generically quantifiable and testable.
- The current NVMe WRR I/O arbitration does not allow that.
- The newly proposed mechanism will enable Linux kernel to support NVMe I/O priority.

Customer-Vendor Requirements Discussions

- Customers provide
  - Several reference workloads
    - Ex 1:
      - High Priority: QD1 4KB Rnd Rd
      - Low Priority: QD1024 4KB 70/30 Rnd Rd/Wr
    - Ex 2:
      - High Priority: 16x Burst 16KB Rnd Rd @ 200ms
      - Medium Priority: QD32 Rnd Rd
      - Low Priority: 10MB/s Rnd Write Rate @ 4KB
    - Ex 3:
      - High Priority: 1x 4KB Rnd Deallocate
      - Low Priority: 100MB/s Seq Write Rate @ 128KB
  - Duration Limit targets. For example:
    - High Priority = 500us @ 99.99%
    - Medium Priority = 1ms @ 99%
    - Low Priority = 10ms @ mean

Advantages

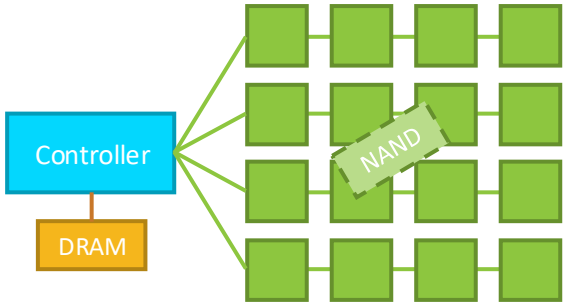
- Vendors design and performance tune for a set of workloads
- Customer discussions on preferred performance tradeoffs for misses

Note: Duration Limit is a requested target completion time.

But reference workloads can pair with a completion percentage achieved by the vendors.

Implement Arbitration Weights to Achieve Duration Limit Targets

- Provided the reference workloads and target duration limits
  - Each SSD Vendor can performance tune a set of arbitration weights for achieving different duration limits
  - Failure to achieve any duration limit targets in the reference workloads can be discussed with the customers
- Dan's *Real World* Predictions
  - Likely: Different arbitration weights for different resources in an SSD
    - Ex: Read path vs Write path
  - Customers will likely make small modifications to the duration limit settings in deployment
    - Vendors will be responsible for linearizing these small changes into interior weight changes



Example SSD Variabilities

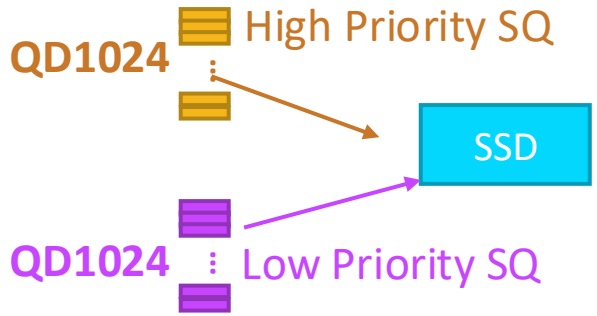
- NAND Speeds
  - Program vs Read changes
  - Generation differences
- DRAM speeds
  - DRAM types
  - Generation differences
- ASIC/Compute Speed
  - Amount of HW offload
  - Fab node differences

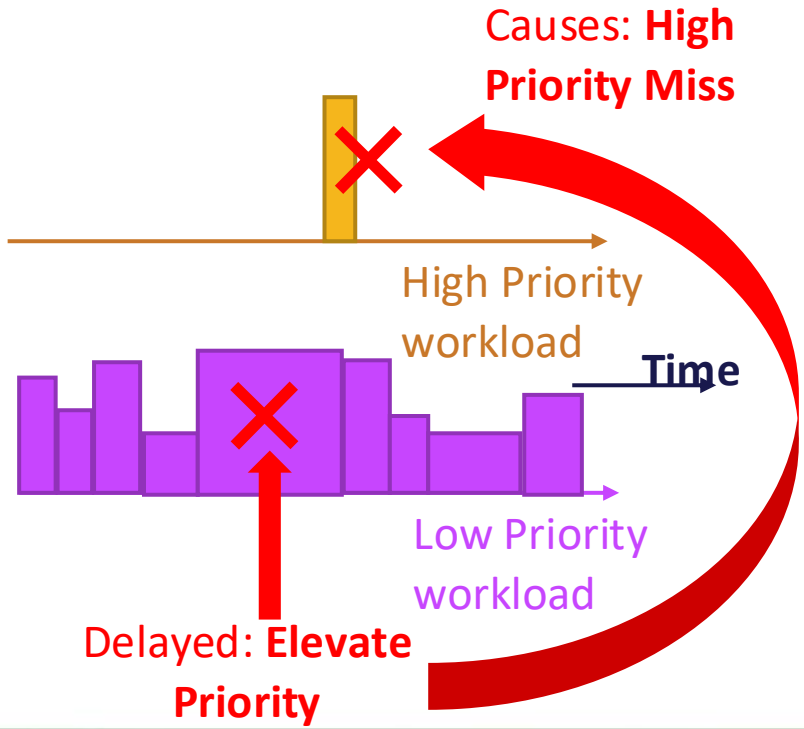
Advantages

- Vendors are responsible for achieving performance parity
- Improved generational and competitor interoperability
- Vendors can test their arbitration weights degrade gracefully for out-of-scope workloads

Avoid – Reactive Architectures

- Possible architecture implementation
  - Recognize Command X is approaching Duration Limit
  - React by racing Command X through at highest priority in order to meet Duration Limit
- Problems with Reactive Architectures
  - What if “all” commands are approaching their Duration Limit?
    - Lose all command differentiation
    - More commands are expected through the lower priority classes
      - This may falsely accelerate the least important commands.
      - Graceful degradation would imply more commands should miss on the low priority classes
  - What if a burst of high priority commands follows 1 slow low priority command?
    - Low priority command should not block the new high priority traffic
    - Reactively accelerating low priority commands
  - What if the low priority command were to hit its own latency issue?
    - Problems above are even worse





Conclusion: Reactive architectures have imperfect knowledge of the future workload.

Do not implement a Reactive Architecture

Call to Action

- IO Prioritization will help improve priority accesses through all stages of the access.

- Let’s align on the details
  - We are looking for input and feedback to holistically solve this problem.

- To get more involved, participate in the discussion in NVM Express!

Open Discussion

